

High-Throughput Lazy Data Streamer

In enterprise-scale data engineering, processing massive datasets requires a "Lazy Evaluation" strategy to prevent system memory exhaustion. By utilizing Python generators to construct chained pipelines, developers can stream millions of records through complex transformations, filtering, and cleaning stages while maintaining a minimal and constant memory footprint.

Implement a modular data processing pipeline using **chained generators**. The system must ingest a raw stream, apply sequential cleaning (whitespace stripping, case normalization), filter out invalid records (**None** values), and perform a final length-based validation, yielding only highly-structured records to the downstream consumer.

Example 1 Input:

```
EXECUTE 4 5
```

Output:

```
{'id': 0, 'raw': 'ITEM_0', 'length': 6} {'id': 2, 'raw': 'ITEM_2',  
'length': 6}
```

Explanation: The source generator produces 4 items. Even-numbered IDs produce valid strings, while odd IDs produce **None**. The pipeline filters the **None** values and processes the remaining strings. Since "ITEM_0" and "ITEM_2" both have a length of 6 (which is ≥ 5), they are yielded as final results.

Example 2 Input:

```
EXECUTE 2 10
```

Output:

```
(No Output)
```

Explanation: The pipeline generates one valid item ("ITEM_0") with length 6. However, since the minimum length threshold is set to 10, the final filter stage discards the item, resulting in an empty output stream.

Function Parameter:

- `limit (int)`: The total number of items to pull from the source generator.
- `min_len (int)`: The minimum string length required to pass the final validation stage.

Returns

- `Iterator[dict]`: A generator yielding cleaned and validated dictionaries.

Constraints

- All pipeline stages (Source, Cleaner, Filter) **must** be implemented as generators using `yield`.
- The source generator must dynamically produce `None` for every odd ID.
- Memory usage must remain constant regardless of the `limit` parameter.

Input Format for Custom Testing Input from stdin contains the `EXECUTE` command followed by the `limit` and `min_len` parameters.

Sample Case 0 Sample Input

```
EXECUTE 1 2
```

Sample Output

```
{'id': 0, 'raw': 'ITEM_0', 'length': 6}
```

Explanation A single record is generated, cleaned, and validated against the length threshold.